ELI — Common Errors and Fixes

ELI v2.1.92

August 2026

Contents

ELI — Common Errors & Fixes

> **Updated for v2.1.51 (August 2026).** The primary install path is now the prebuilt, CI-launch-tested installers on GitHub Releases (Windows Setup.exe, macOS dmg, Linux AppImage) with first-boot GPU (CUDA/Vulkan/Metal) and starter-model offers; data lives in a per-user `ELI_v2` folder that survives upgrades. Source installs below remain fully supported.

A quick reference for the gremlins that actually come up when running ELI. Each one is symptom → cause → the exact fix. Add new ones as they surface.

Clicking a link opens no browser page

Symptom: the server prints the URLs fine and copy-paste works, but Ctrl/right-click → "Open Link" opens nothing. **Cause:** broken **snap Firefox** — a `mesa-2404` auto-refresh leaves Firefox's GPU content-mount stale (`/snap/firefox/…/gpu-2404-provider-wrapper` goes missing), so *launching* a fresh Firefox fails silently while an already-open window still works. **Fix:**

```
pkill -9 firefox
sudo snap disconnect firefox:gpu-2404
sudo snap connect firefox:gpu-2404 mesa-2404:gpu-2404
# if it's still stuck:
sudo snap refresh firefox
sudo snap refresh mesa-2404
```

Reopen Firefox once, then relaunch the server — clicks (and the auto-open) work again. *(Nothing wrong with ELI — the URLs and server are fine; it's the snap.)*

Wrong / tiny model loads instead of your usual one

Symptom: ELI boots on a small model (e.g. tinyllama-1B) even though settings point at your 35B A3B. **Cause:** a stray `ELI_GGUF_MODEL_PATH` / `ELI_MODEL_PATH` env var in your shell **overrides config** (it wins over everything). **Fix:**

```
echo $ELI_GGUF_MODEL_PATH        # if this prints a model path, that's the culprit
unset ELI_GGUF_MODEL_PATH ELI_MODEL_PATH
```

Or just switch live from the dashboard: **Settings → Model**.

Server won't start / "address already in use"

Cause: a previous instance is still bound to the port. **Fix:**

```
pkill -f "api/server.py"      # or find it: ss -ltnp | grep 8081
```

Then relaunch. ELI's web port is **8081**, HTTPS **8443**.

Phone can't connect after a restart

Symptom: a paired phone gets 401 / "can't access the server." **Cause:** it's carrying an old token (server restarted, or you rotated the token). **Fix:** the token now **persists** across restarts — just re-open the current link/QR from the **Connect** tab. To deliberately kick a lost phone off, hit **rotate** and re-pair.

Phone can't connect at all (fresh pairing)

Cause: the firewall is blocking the port. **Fix:** run the exact `sudo ufw allow …` lines the server prints on startup, and make sure the phone is on the **same Wi-Fi**.

Phone microphone / voice doesn't work

Cause: browsers block the mic (`getUserMedia`) on a plain `http://LAN-IP` page. **Fix:** start with `--https` and open the `https://…:8443/` link on the phone; accept the one-time self-signed "not private" warning.

Model dropdown in the dashboard is empty

Cause: the list endpoint was being shadowed by the OpenAI-compatible `/v1/models` route. **Fix:** already fixed — the list lives at `/v1/models/installed`. If a fork regresses it, make sure the dashboard fetches that path, not `/v1/models`.

News shows old stories as "the latest"

Cause: the interest-matched half wasn't recency-gated. **Fix:** already fixed (a freshness gate drops stale niche matches). If it recurs, say `refresh the news`.

Terminal shows escape-code junk around a URL (`^[]8;;…`)

Cause: you're looking at a **log file** or `cat -v` output — those *always* render raw escape codes. A live terminal renders the link normally. Not a bug.

`git push` → "Could not resolve host github.com"

Cause: a transient DNS/network blip (ELI is offline-by-default, but git is separate). **Fix:** just retry the push.

Vision / CLIP segfaults on the GPU

Cause: the CLIP/vision path can segfault on some GPUs. **Fix:** CLIP runs on **CPU** by design; keep it there. Main-model + vision hot-swap handle VRAM automatically — don't force CLIP onto the GPU.

Arch / lean distros: "attempt to write a readonly database" (portable install)

Symptom: on the portable Linux build, first-run DB init (Step 5) fails for the `user.*` stores with `sqlite3.OperationalError: attempt to write a readonly database`, while `system_index` / `coding_memory` / `agent` succeed in the same folder; the GUI then can't open the memory DB and exits. **Cause:** the folder you extracted ELI into lives on a filesystem whose locking SQLite's **WAL journal** needs isn't supported — **NTFS, exFAT/FAT, or a network mount** (very common for a `~/Downloads` on a dual-boot data drive). WAL returns `SQLITE_READONLY` there; the rollback-journal stores beside it work fine, which is the tell-tale split. (This is also why the AppImage — which stores data under `~/.local/share/ELI_v2` on your main partition — worked while the portable build in `~/Downloads` didn't.) **Fix:** fixed in **v2.1.19** — ELI detects a WAL-hostile filesystem and falls back to a DELETE (rollback) journal automatically, so it runs wherever you put it. On an older build, extract ELI onto an **ext4/btrfs** path (e.g. under your home partition) instead. Confirm your mount with: `findmnt -T . -o TARGET,FSTYPE`.

Arch / lean distros: GUI won't open — "libxcb-cursor0 is needed to load the Qt xcb platform plugin"

Symptom: the AppImage's engine loads (you see the `ELI v2.0` banner and the module log) but the window never appears; it aborts with the `xcb-cursor0 … xcb platform plugin` message. **Cause:** Qt 6.5+'s xcb platform plugin (`libqxcb.so`) links the **whole xcb-util family** — `libxcb-cursor`, `libxcb-icccm`, `libxcb-image`, `libxcb-keysyms`, `libxcb-render-util`, `libxcb-util` — which a minimal desktop doesn't install. **Qt's message is misleading:** it always blames `libxcb-cursor0` even when the real missing library is a different member (on a test Arch box the actual culprit was `libxcb-icccm.so.4`). **Fix:** fixed in **v2.1.21** — the AppImage now bundles the full xcb-util family (in `PySide6/Qt/lib`), so it launches out of the box on bare Arch/Debian with no extra packages. On an older build, install them yourself:

```
# Arch
sudo pacman -S xcb-util-cursor xcb-util-wm xcb-util-image xcb-util-keysyms xcb-util-renderutil xcb-util
# Debian/Ubuntu
sudo apt install libxcb-cursor0 libxcb-icccm4 libxcb-image0 libxcb-keysyms1 libxcb-render-util0 libxcb-util1
```

Diagnose exactly which library is missing (Qt's own trace names it):

```
QT_DEBUG_PLUGINS=1 ./ELI_v2-*-x86_64.AppImage 2>&1 | grep -iE 'cannot|not found'
```

Running ELI on Arch — verified working steps

The **AppImage** is the easiest path: it bundles its own **Python 3.11**, so Arch's system Python 3.14 (which has no `llama-cpp-python` wheel) is irrelevant, and as of **v2.1.21** it bundles every Qt xcb library too. Download and run:

```
U=https://github.com/ShadowESC95/ELI_v2.0/releases/download/v2.1.51
wget "$U/ELI_v2-2.1.51-x86_64.AppImage"
chmod +x ELI_v2-2.1.51-x86_64.AppImage
./ELI_v2-2.1.51-x86_64.AppImage
```

Run it directly (as above) so it FUSE-mounts in place — `--appimage-extract-and-run` unpacks ~4 GB into `/tmp`, which fails with `libz.so.1: file too short` when `/tmp` is a small `tmpfs`. If you launch it over **SSH** into a running desktop session, prefix the command with `export DISPLAY=:0`. (Paste long URLs as a **single line** — a wrapped URL 404s and the tail runs as a stray command.)

Verified end-to-end on a clean Arch VM (XFCE): engine loads, database initialises on a WAL-hostile disk via the DELETE fallback, and the GUI opens with **every** xcb-util package uninstalled.

"ELI can't see my Ollama models" (any OS)

Symptom: Backend is set to Ollama but the model list is empty, or ELI reports Ollama unreachable (`[Errno 111] Connection refused`) while Ollama is installed.

Fix (v2.1.30): ELI now starts Ollama for you. The #1 cause is simply that the local Ollama server isn't running. If the `ollama` program is installed but stopped, ELI now runs `ollama serve` automatically when you pick/refresh Ollama, waits for it, and loads your models — no terminal needed. If it still can't start (Ollama not installed, or a remote host), you get per-OS guidance. Disable the auto-start with `ELI_OLLAMA_AUTOSTART=0`. Full walkthrough for non-technical users: `blueprints/ollama_startup_guide.md`.

Fix (v2.1.31): the toolbar dropdown itself was broken. Separately from the server ever starting, the toolbar's Ollama dropdown could sit permanently empty with the status dot stuck on "Checking Ollama…" *even when Ollama was running and serving models perfectly*. The widget fetched the list on a background thread and handed the result back with `QTimer.singleShot(0, …)` — but a Qt timer started

from a plain `threading.Thread` has no Qt event loop to run on, so the callback was silently dropped and the combo was never filled. It now hands results back over a Qt **signal**, which queues to the GUI thread correctly. The model-pull progress dialog had the same defect (progress never advanced, dialog never closed) and was fixed the same way. Regression-tested in `tests/test_ollama_selector_threading.py`, including a guard that fails if a future refactor reintroduces a timer on that path. Note this was invisible to the existing GUI tests because they only asserted the widget *constructs* — the new tests assert models actually land in the dropdown.

Earlier address-handling fixes (still in place) — you no longer need to do anything special about how the host is written:

- **`OLLAMA_HOST` set the way Ollama documents it** (`OLLAMA_HOST=127.0.0.1:11434`, with no `http://`) used to break ELI with `unknown url type`. Scheme-less hosts, bare IPs and missing ports are now all understood, everywhere — client, startup picker, wizard, Settings.
- **`localhost` resolving to IPv6 first** (common on Windows) while Ollama listens on IPv4 — ELI now automatically retries `127.0.0.1`, so this no longer looks like "Ollama is down".
- **Ollama on another machine** was blocked by ELI's offline-by-default guard with no explanation. A host you configure is now treated as a deliberate local service, like a LAN MQTT broker, so it is allowed and the Net toggle can stay OFF.

If it is still not found:

```
ollama list                    # is it running, and does it have models?
ollama pull llama3.2           # pull one if the list is empty
```

Per-OS start: **Windows/macOS** — Ollama starts itself (tray / menu bar); launch "Ollama" if it isn't running. **Linux** — `systemctl --user start ollama`, or `ollama serve`.

For Ollama on a **different machine**, on that machine:

```
OLLAMA_HOST=0.0.0.0:11434 ollama serve   # listen beyond localhost
```

and allow port `11434` through its firewall. Then in ELI set **Settings → Model → Ollama → Host** to that machine (e.g. `192.168.1.20` — the port is filled in for you).

ELI dies mid-session: `double free or corruption` / `Aborted (core dumped)`

Symptom: the GUI is running fine, then the terminal prints a run of `Warning: maximum number 350 of (N_VOICES_LIST = 350 - 1) reached`, followed by `double free or corruption (!prev)` and `Aborted (core dumped)`. The whole app is gone.

Cause: not ELI. espeak-ng has a hardcoded `N_VOICES_LIST` of 350. On a machine with more installed voice/variant combinations than that, `espeak_ListVoices` walks off the end of its array and corrupts the heap. ELI reached it through `pyttsx3.init()` while listing system voices. Because that is a **native** `abort()`, the Python `try/except` around it caught nothing — the crash took the entire GUI with it.

Fix (v2.1.32): system-voice enumeration now runs in a **subprocess**. If espeak crashes, it kills a throw-away child; ELI logs the non-zero exit and continues with no system voices listed. Nothing to configure.

The terminal floods with `Audio source must be entered before listening`

Symptom: thousands of identical `[AUDIO] Listen error: Audio source must be entered before listening…` lines, a pinned CPU core, and a log that grows without limit.

Cause: `listen_once()` and the mic-calibration helper open `with self.microphone` on the **same** Microphone object the background listen loop is holding. `speech_recognition`'s `__exit__` sets `stream = None`, so when either finishes it closed the stream out from under the running loop — which then failed on every iteration, forever, at full speed.

Fix (v2.1.32): the loop detects the closed stream and **re-enters** it instead of spinning, and that error is rate-limited (once per 5s with a count) so no fault can bury the log again.

ELI reports facts about you, then calls them a hallucination when you push back

Symptom: ELI gives a grounded profile answer, you say "wrong", and it retracts everything — "that was likely a hallucination", "I don't have access to your personal files", "I only see the text in this window".

Cause: those statements are **false**. The facts came out of `user.sqlite3`. This is the base model's generic cloud-assistant reflex overriding what ELI actually is. There was a rule against *inventing* mechanisms, but none against *denying real ones* — the mirror failure.

Fix (v2.1.32): a `NO FALSE SELF-DENIAL` rule. Push-back is not proof ELI was wrong: it must correct the **specific** disputed field and say where the stored value came from, never retract a whole grounded answer as imaginary. The phrases also now trip the evidence validator. A companion rule stops "what do you know about yourself" being answered with *your* profile.

A reply ends abruptly on a dangling -

Symptom: a list-shaped answer stops mid-structure with an empty bullet on the last line.

Cause: the generator hit its token cap just as it opened the next bullet. The truncation detector missed it (a - is neither alphanumeric nor a sentence terminator, so it read as a clean ending), and the re-generation repair is deliberately **non-quick only** — a second inference would defeat quick mode's latency.

Fix (v2.1.32): the detector now recognises an orphan list marker, and the output governor trims it in **every** mode, so a capped answer ends on its last complete line.

"ELI refuses a general question with 'the net's off, I can't verify that' "

Symptom: an everyday advice/how-to question — *"what is the best way to eat breakfast"* — is met with *"I can't verify that right now — the net's off … turn the Net toggle on"*, and with the Net on it returns an irrelevant web snippet (a user saw *"Best Buy doesn't offer breakfast recommendations"*).

Cause: the fact classifier matched the "**what … is**" shape and treated the advice question as a checkable external fact (oddly, the apostrophe form *"what's the best way…"* slipped through and answered normally). So offline it hit the honest can't-verify hedge, and online it was web-searched — surfacing whatever the search returned.

Fix: two layers, no action needed on your part. - **v2.1.25** — advice / how-to / recommendation questions ("best way to…", "how do I…", "how to…", "what should I…", "tips for…", "should I…") are no longer classed as external facts. They're answered directly from the model's own knowledge, offline, like any other chat. Genuine facts ("what is the capital of Japan", "how old is …", "who won …") still verify as before. - **v2.1.26** — a *relevance gate* on web results: when a genuine fact is searched and the search returns topically-unrelated pages (the "Best Buy" case), ELI no longer synthesises them into a confident non-answer. It says it searched but couldn't find anything that answers the question, and asks you to rephrase or add a detail — an honest miss instead of a made-up one. Tune with `ELI_WEB_RELEVANCE_FLOOR` (default `0.34`; higher = stricter).

Natural voice / cloning: `pip install "eli-v2.0[natural]"` fails with "No matching distribution"

Symptom: following an in-app message or older doc literally (`pip install "eli-v2.0[natural]"`) fails — pip tries to fetch `eli-v2.0` from PyPI, where this project isn't published. **Cause:** that string only resolves once `eli-v2.0` is already installed under that name from a real index; a source checkout needs the local-path form instead. **Fix:** from the ELI project root: `pip install -e ".[natural]"` (or `.[clone]`/`.[voice]` — all three are aliases for the same Coqui XTTS-v2 extra). Fixed at the source in v2.1.29 — every in-app message now shows the correct form.

Natural voice / cloning: `ImportError: cannot import name 'isin_mps_friendly'`

Symptom: after installing the `[natural]`/`[clone]` extra, the first clone/natural-voice attempt raises `ImportError: cannot import name 'isin_mps_friendly' from 'transformers.pytorch_utils'`. **Cause:** `coqui-tts` (latest release, 0.27.5) still calls a `transformers` helper that existed only as an old Apple-MPS `torch.isin()` fallback and was removed in `transformers>=5` — the exact version ELI's own lock file pins for the rest of the stack, so downgrading transformers isn't an option. **Fix:** fixed in v2.1.29 — `eli/perception/tts_xtts.py` restores the function with a lazy compat shim (plain `torch.isin()`, identical behaviour off MPS) before `TTS` is imported. No action needed on an up-to-date checkout.

Natural voice / cloning: hangs or `EOFError` on first use

Symptom: the very first clone or natural-voice synthesis after installing the extra either hangs indefinitely or raises `EOFError: EOF when reading a line`, and the GUI/voice command silently falls back to a normal voice instead. **Cause:** `coqui-tts` gates the first XTTS-v2 model download behind an interactive y/n terminal prompt (agree to the non-commercial Coqui Public Model License, or confirm a paid licence). ELI's GUI has no terminal for that prompt to read from. **Fix:** fixed in v2.1.29 — `tts_xtts._get_model()` sets `COQUI_TOS_AGREED=1` before loading, which is always the correct branch here (a local, personal voice, never redistributed by ELI). No action needed on an up-to-date checkout; using this feature at all implies agreeing to the non-commercial CPML (a commercial deployment needs a separate paid licence directly from Coqui).

Natural voice / cloning: `ModuleNotFoundError: No module named 'torchaudio'` (or `torchcodec`)

Symptom: `xtts_available()` / a clone attempt fails with a missing `torchaudio` or `torchcodec` import, even though `coqui-tts` installed successfully. **Cause:** `coqui-tts` doesn't declare `torch`/`torchaudio`/`torchcodec` as install dependencies (same reasoning as ELI's own `training` extra — a GPU build must match your CUDA, so it can't be hard-pinned), but genuinely needs all three at import time. **Fix:** fixed in v2.1.29 — the `clone`/`natural`/`voice` extras now pull in `torch`, `torchaudio`, and `torchcodec` themselves, so a plain `pip install -e ".[natural]"` on a fresh machine resolves everything together. If you already had a bleeding-edge `torch` installed *ahead* of the latest published `torchaudio`/`torchcodec` (e.g. a nightly build), install the matching version manually from `https://download.pytorch.org/whl/<your-cuda-tag>` — same pattern as the main `torch` line in `requirements.txt`.

---

*House rule: when a new error bites, add it here — symptom, cause, and the **exact** commands that fixed it. Future-you will thank present-you.*